AI Chat Proxy — Improvement Recommendations

Project: AI Chat Proxy
Technology: JavaScript, Node.js, Express, OpenAI API
Location: `/home/dominic/Documents/ai-chat-proxy`
Report #: 19/20

Short-term

- Add request validation middleware
- Add structured logging
- Add response caching (in-memory TTL)

Medium-term

- Add per-user API keys
- Add usage tracking
- Add request/response logging

Long-term

- Add model fallback (GPT-4 → GPT-3.5)
- Add admin dashboard
- Add multi-model routing

Improvement Recommendations

Priority Matrix

Priority	Effort	Impact	Recommendation
P1	Medium	High	PostgreSQL migration
P2	Low	High	Rate limiting for all
P3	Medium	High	CI/CD pipeline
P4	Low	Medium	Security headers
P5	High	Medium	Frontend tests

Critical Improvements

1. Database Migration to PostgreSQL

- **Effort:** Medium
- **Impact:** High
- **Details:** SQLite is not suitable for production workloads. Migrate to PostgreSQL using Alembic for schema versioning.
- **Steps:**
- Install psycopg2
- Update DATABASE_URL
- Create Alembic configuration
- Generate initial migration
- Test with PostgreSQL locally
- Update Render deployment

2. Add Comprehensive Rate Limiting

- **Effort:** Low
- **Impact:** High
- **Details:** Only AI Chat Proxy has rate limiting. Add to all services to prevent abuse.
- **Implementation:** FastAPI middleware with Redis backend (or in-memory for simplicity)

3. Implement CI/CD Pipeline

- **Effort:** Medium
- **Impact:** High
- **Details:** GitHub Actions for running tests and auto-deploying to Render on merge to main.

Recommended Features

- Admin dashboard with usage analytics
- API key management for third-party access
- Webhook notifications on task completion
- Batch URL shortening
- RSS feed for blog posts

Refactoring Opportunities

1. Extract shared auth logic into a common package
2. Standardize error response format across all services
3. Move configuration to environment variables for all secrets
4. Add OpenAPI documentation for all endpoints
5. Implement health check aggregation service